Comment prédire la consommation électrique

Les modèles XGBoost, LightGBM et CatBoost, expliqués simplement

Projet S9 POWER · encadrants : E. Grivel, A. Leys

1 **Notre projet**
ce que nous cherchons à prédire, et pourquoi cette méthode

2 **Les bases**
l'arbre de décision, puis le boosting : des arbres qui se corrigent

3 **Trois modèles**
XGBoost, LightGBM, CatBoost : leur nom, leur intérêt, leur rôle chez nous

4 **Les comparer**
notre méthode d'expérience

5 **Base de données et encodage**
notre table horaire, puis l'encodage des catégories

Notre projet et nos données

POWER : prévoir la consommation électrique française, heure par heure.

Peut-on prévoir ce que la France va consommer demain ?

Pourquoi : l'électricité ne se stocke pas

Repère : prévision de RTE, $\approx$ 2 % d'erreur

Exemple : 12 oct. 2026, 16 h · 46 648 MW réels · 45 800 MW prévus par RTE

Notre projet et nos données

POWER : prévoir la consommation électrique française, heure par heure.

Peut-on prévoir ce que la France va consommer demain ?

Pourquoi : l'électricité ne se stocke pas

Repère : prévision de RTE, $\approx$ 2 % d'erreur

Exemple : 12 oct. 2026, 16 h · 46 648 MW réels · 45 800 MW prévus par RTE

Indicateurs : connus à l'avance				Cible
Date et heure	Jour	Temp.	Férié	Conso (MW)
14 janv. 2025, 03:00	mardi	1 °C	non	**56 200**
14 janv. 2025, 19:00	mardi	2 °C	non	**78 100**
19 janv. 2025, 19:00	dimanche	4 °C	non	**66 300**
15 août 2025, 13:00	vendredi	28 °C	oui	**36 500**
…	…	…	…	**…**

Valeurs illustratives · 1 ligne = 1 heure

Notre projet et nos données

POWER : prévoir la consommation électrique française, heure par heure.

Peut-on prévoir ce que la France va consommer demain ?

Pourquoi : l'électricité ne se stocke pas

Repère : prévision de RTE, $\approx$ 2 % d'erreur

Exemple : 12 oct. 2026, 16 h · 46 648 MW réels · 45 800 MW prévus par RTE

Indicateurs : connus à l'avance				Cible
Date et heure	Jour	Temp.	Férié	Conso (MW)
14 janv. 2025, 03:00	mardi	1 °C	non	**56 200**
14 janv. 2025, 19:00	mardi	2 °C	non	**78 100**
19 janv. 2025, 19:00	dimanche	4 °C	non	**66 300**
15 août 2025, 13:00	vendredi	28 °C	oui	**36 500**
…	…	…	…	**…**

Valeurs illustratives · 1 ligne = 1 heure

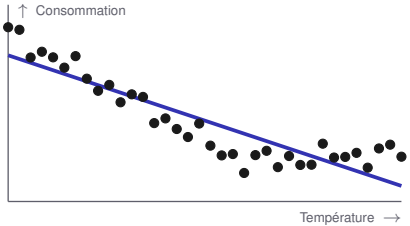
1 Collecter consommation, météo, calendrier › **2 Assembler** une table : 1 ligne = 1 heure › **3 Entraîner** le modèle apprend sur le passé › **4 Prédire** une année jamais vue › **5 Comparer** à la réalité et à RTE

Entraîner = montrer des milliers d'exemples dont on connaît déjà la réponse

Pourquoi cette méthode, et pas une autre ?

Plus il fait froid, plus on chauffe, puis la courbe devient plate.

Consommation selon la température

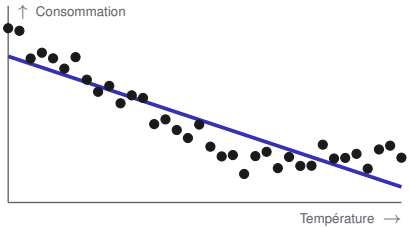


En bleu : une droite. Elle se trompe presque partout.

Pourquoi cette méthode, et pas une autre ?

Plus il fait froid, plus on chauffe, puis la courbe devient plate.

Consommation selon la température



En bleu : une droite. Elle se trompe presque partout.

Les autres pistes

Régression linéaire

Idée : une droite

Limite : courbe coudée ratée

Séries temporelles (ARIMA)

Idée : prolonger une courbe qui se répète

Limite : peu adapté à la météo

Réseaux de neurones (LSTM)

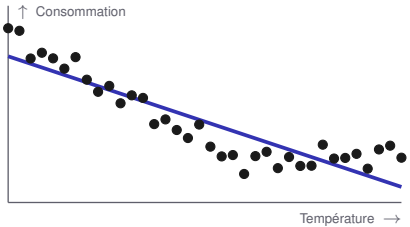
Idée : un « cerveau artificiel »

Limite : gourmand en données, peu explicable

Pourquoi cette méthode, et pas une autre ?

Plus il fait froid, plus on chauffe, puis la courbe devient plate.

Consommation selon la température



En bleu : une droite. Elle se trompe presque partout.

Les autres pistes

Régression linéaire

Idée : une droite

Limite : courbe coudée ratée

Séries temporelles (ARIMA)

Idée : prolonger une courbe qui se répète

Limite : peu adapté à la météo

Réseaux de neurones (LSTM)

Idée : un « cerveau artificiel »

Limite : gourmand en données, peu explicable

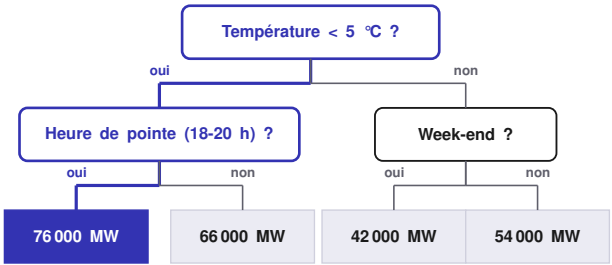
Notre choix

Arbres de décision + boosting : suivent les courbes, mélangent heure, météo, calendrier, restent explicables.

L'arbre de décision : un jeu de questions

Une suite de questions oui / non, puis une prédiction.

Exemple : mardi, 19 h, 2 °C

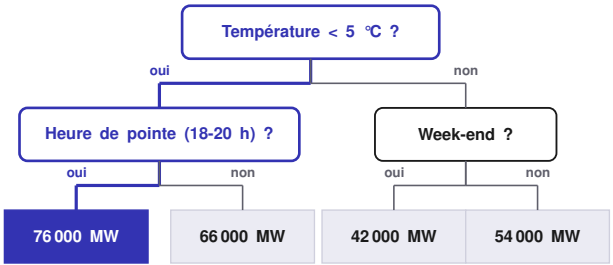


Chemin en bleu : 2 °C, 19 h → 76 000 MW (chiffres illustratifs).

L'arbre de décision : un jeu de questions

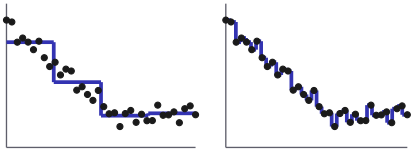
Une suite de questions oui / non, puis une prédiction.

Exemple : mardi, 19 h, 2 °C



Chemin en bleu : 2 °C, 19 h → 76 000 MW (chiffres illustratifs).

Comment il apprend : il découpe



4 groupes ✓ : suit la forme

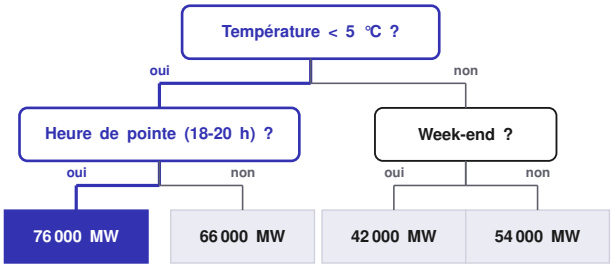
30 groupes × : par cœur

Chaque feuille : moyenne des cas passés semblables.

L'arbre de décision : un jeu de questions

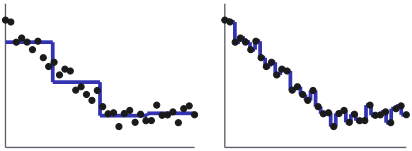
Une suite de questions oui / non, puis une prédiction.

Exemple : mardi, 19 h, 2 °C



Chemin en bleu : 2 °C, 19 h → 76 000 MW (chiffres illustratifs).

Comment il apprend : il découpe



4 groupes ✓ : suit la forme

30 groupes × : par cœur

Chaque feuille : moyenne des cas passés semblables.

Un seul arbre ne suffit pas

Trop simple, ou par cœur. L'idée du boosting : en combiner beaucoup de petits.

Le boosting : des arbres qui se corrigent

Un mardi à 19 h par grand froid : 78 000 MW réels (chiffres illustratifs).

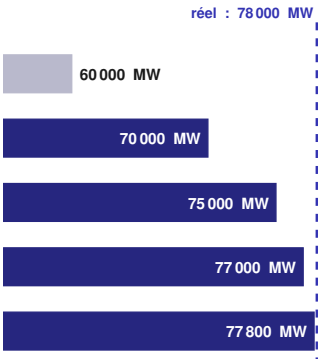
Arbre 1
la moyenne de toutes les heures

+ Arbre 2
« moins de 5 °C » : +10 000

+ Arbre 3
« heure de pointe » : +5 000

+ Arbre 4
« mardi d'hiver » : +2 000

+ Arbre 5 et suivants
petits ajustements



Échelle à partir de 55 000 MW.

Le boosting : des arbres qui se corrigent

Un mardi à 19 h par grand froid : 78 000 MW réels (chiffres illustratifs).

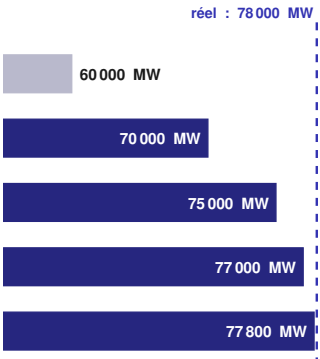
Arbre 1
la moyenne de toutes les heures

+ Arbre 2
« moins de 5 °C » : +10 000

+ Arbre 3
« heure de pointe » : +5 000

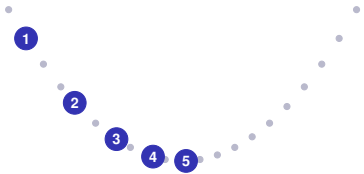
+ Arbre 4
« mardi d'hiver » : +2 000

+ Arbre 5 et suivants
petits ajustements



Échelle à partir de 55 000 MW.

Pourquoi « gradient » ? La pente

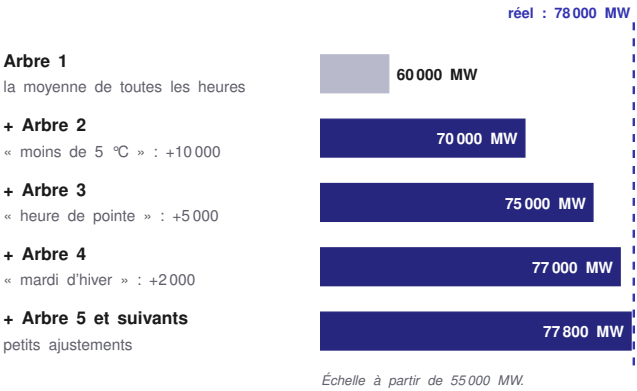


Chaque numéro = un arbre = un pas vers le bas.

Gradient = la pente
Learning rate = taille du pas
Nombre d'arbres = nombre de pas

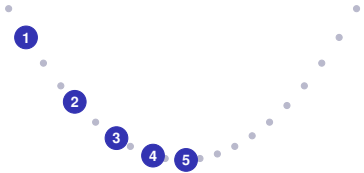
Le boosting : des arbres qui se corrigent

Un mardi à 19 h par grand froid : 78 000 MW réels (chiffres illustratifs).



Échelle à partir de 55 000 MW.

Pourquoi « gradient » ? La pente



Chaque numéro = un arbre = un pas vers le bas.

Gradient = la pente
Learning rate = taille du pas
Nombre d'arbres = nombre de pas

Chaque arbre corrige le reste

Un arbre = faible · quelques centaines = très précis. To boost = renforcer.

XGBoost : principe et prédictions

eXtreme Gradient Boosting · des arbres de décision entraînés les uns après les autres

Idée fondamentale

Chaque nouvel arbre apporte une **correction** à la prédiction actuelle.

$$\hat{y}_i^{(T)} = \hat{y}_i^{(0)} + \sum_{t=1}^{T} \eta \, f_t(x_i)$$

départ + corrections des T arbres

XGBoost : principe et prédictions

eXtreme Gradient Boosting · des arbres de décision entraînés les uns après les autres

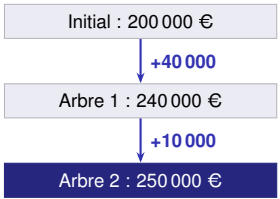
Idée fondamentale

Chaque nouvel arbre apporte une **correction** à la prédiction actuelle.

$$\hat{y}_i^{(T)} = \hat{y}_i^{(0)} + \sum_{t=1}^{T} \eta \, f_t(x_i)$$

départ + corrections des T arbres

Exemple : chaque arbre corrige



XGBoost : principe et prédictions

eXtreme Gradient Boosting · des arbres de décision entraînés les uns après les autres

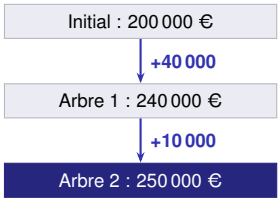
Idée fondamentale

Chaque nouvel arbre apporte une **correction** à la prédiction actuelle.

$$\hat{y}_i^{(T)} = \hat{y}_i^{(0)} + \sum_{t=1}^{T} \eta \, f_t(x_i)$$

départ + corrections des T arbres

Exemple : chaque arbre corrige



Learning rate η

Il réduit la taille des corrections : avec $\eta = 0{,}1$, une correction de 40 000 € ne compte que pour 4 000 €.

Comment XGBoost construit ses arbres ?

La perte est approchée par un développement de Taylor d'ordre 2.

Gradient et hessienne de la perte

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i} \qquad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

Pour une feuille j, on additionne ceux de ses lignes :

$$G_j = \sum_{i \in j} g_i \qquad H_j = \sum_{i \in j} h_i$$

Comment XGBoost construit ses arbres ?

La perte est approchée par un développement de Taylor d'ordre 2.

Gradient et hessienne de la perte

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i} \qquad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

Pour une feuille j, on additionne ceux de ses lignes :

$$G_j = \sum_{i \in j} g_i \qquad H_j = \sum_{i \in j} h_i$$

Poids d'une feuille et gain d'une coupure

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

$$\text{Gain} = \frac{1}{2}\left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda}\right) - \gamma$$

L, R : les deux feuilles créées · P : la feuille coupée

Comment XGBoost construit ses arbres ?

La perte est approchée par un développement de Taylor d'ordre 2.

Gradient et hessienne de la perte

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i} \qquad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

Pour une feuille j, on additionne ceux de ses lignes :

$$G_j = \sum_{i \in j} g_i \qquad H_j = \sum_{i \in j} h_i$$

Poids d'une feuille et gain d'une coupure

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

$$\text{Gain} = \frac{1}{2}\left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda}\right) - \gamma$$

L, R : les deux feuilles créées · P : la feuille coupée

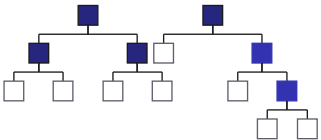
Interprétation

λ limite les poids des feuilles ; γ pénalise chaque nouvelle coupure. L'algorithme teste les coupures possibles et garde, une à une, celles dont le gain est favorable.

LightGBM : le boosting « léger »

Light Gradient Boosting Machine · Microsoft, 2016 · « light » = rapide et léger

1. Il coupe feuille par feuille



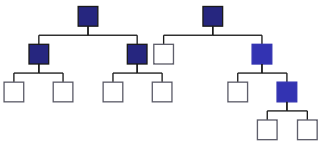
par niveau **par feuille**

Il ne creuse que la feuille où l'erreur est la plus forte.

LightGBM : le boosting « léger »

Light Gradient Boosting Machine · Microsoft, 2016 · « light » = rapide et léger

1. Il coupe feuille par feuille



par niveau **par feuille**

Il ne creuse que la feuille où l'erreur est la plus forte.

2. Il range les valeurs en tiroirs

sans tiroirs : chaque valeur



avec tiroirs (environ 255) : les frontières

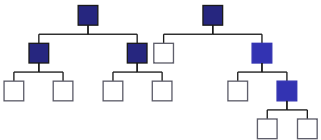


Peu de seuils à tester : bien plus rapide.

LightGBM : le boosting « léger »

Light Gradient Boosting Machine · Microsoft, 2016 · « light » = rapide et léger

1. Il coupe feuille par feuille



Il ne creuse que la feuille où l'erreur est la plus forte.

2. Il range les valeurs en tiroirs

sans tiroirs : chaque valeur



avec tiroirs (environ 255) : les frontières



Peu de seuils à tester : bien plus rapide.

3. Il se concentre sur ses erreurs



GOSS : il apprend surtout sur ce qu'il rate encore.

En bref

Même logique que XGBoost, mais un entraînement nettement plus rapide et moins gourmand en mémoire.

LightGBM dans notre projet

Notre modèle pour tester vite et donner une fourchette.

1. Tester les indicateurs

Base : heure, jour, mois

\+ température

\+ jours fériés

\+ vacances

On garde un indicateur s'il fait baisser l'erreur sur 2024.

LightGBM dans notre projet

Notre modèle pour tester vite et donner une fourchette.

1. Tester les indicateurs

Base : heure, jour, mois

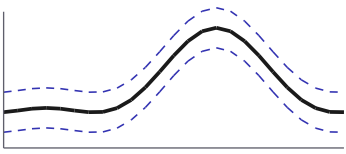
\+ température

\+ jours fériés

\+ vacances

On garde un indicateur s'il fait baisser l'erreur sur 2024.

2. Une fourchette de valeurs



Quantiles 10 et 90 % : « entre 70 000 et 80 000 MW ».

LightGBM dans notre projet

Notre modèle pour tester vite et donner une fourchette.

1. Tester les indicateurs

Base : heure, jour, mois

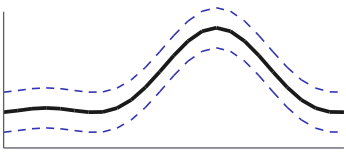
\+ température

\+ jours fériés

\+ vacances

On garde un indicateur s'il fait baisser l'erreur sur 2024.

2. Une fourchette de valeurs



Quantiles 10 et 90 % : « entre 70 000 et 80 000 MW ».

3. Ses pièges

! Trop de feuilles : par cœur

! Record de froid : sous-estimé

! Importance : lire le gain

Parade : arrêt anticipé sur 2024.

Son rôle chez nous

Explorer vite les indicateurs, et donner des fourchettes au tableau de bord.

CatBoost : le boosting des catégories

Categorical Boosting · « cat » = catégorie · Yandex, 2017

1. Il lit directement les catégories

Sans : 7 colonnes 0 / 1

1	0	0	0	0	0	0
0	**1**	0	0	0	0	0
0	0	**1**	0	0	0	0

lun · mar · mer…

Avec : des mots

lundi
mardi
…

Jour, mois, zone : lus tels quels, sans passer en nombres.

CatBoost : le boosting des catégories

Categorical Boosting · « cat » = catégorie · Yandex, 2017

1. Il lit directement les catégories

Sans : 7 colonnes 0 / 1

1	0	0	0	0	0	0
0	**1**	0	0	0	0	0
0	0	**1**	0	0	0	0

lun · mar · mer…

Avec : des mots

lundi
mardi
…

Jour, mois, zone : lus tels quels, sans passer en nombres.

2. Il évite de tricher



Pour la ligne 5, il n'utilise que les lignes 1 à 4.

Une ligne n'est jamais corrigée par un modèle qui l'a déjà vue.

CatBoost : le boosting des catégories

Categorical Boosting · « cat » = catégorie · Yandex, 2017

1. Il lit directement les catégories

Sans : 7 colonnes 0 / 1

1	0	0	0	0	0	0
0	**1**	0	0	0	0	0
0	0	**1**	0	0	0	0

lun · mar · mer…

Avec : des mots

lundi
mardi
…

Jour, mois, zone : lus tels quels, sans passer en nombres.

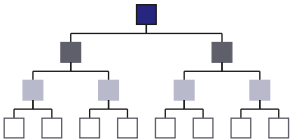
2. Il évite de tricher



Pour la ligne 5, il n'utilise que les lignes 1 à 4.

Une ligne n'est jamais corrigée par un modèle qui l'a déjà vue.

3. Des arbres symétriques



même question sur tout un étage

Même question à chaque étage : prédiction très rapide.

En bref

Peu de réglages ; entraînement souvent plus lent que LightGBM.

CatBoost dans notre projet

Notre modèle pour les variables qui sont des mots.

Nos colonnes : nombres ou catégories ?

heure	nombre	19
jour de la semaine	catégorie	mardi
mois	catégorie	janvier
férié	catégorie	oui / non
zone de vacances	catégorie	A, B ou C
température	nombre	2 °C
consommation 7 jours avant	nombre	74 500 MW

Valeurs d'exemple illustratives.

CatBoost dans notre projet

Notre modèle pour les variables qui sont des mots.

Nos colonnes : nombres ou catégories ?

heure	nombre	19
jour de la semaine	catégorie	mardi
mois	catégorie	janvier
férié	catégorie	oui / non
zone de vacances	catégorie	A, B ou C
température	nombre	2 °C
consommation 7 jours avant	nombre	74 500 MW

Valeurs d'exemple illustratives.

Comment on l'utilise

On indique les colonnes catégorielles :

```
cat_features = [jour, mois, férié, zone]
```

- Mots lus tels quels.
- Bon résultat sans long réglage.
- Plus lent à entraîner.

CatBoost dans notre projet

Notre modèle pour les variables qui sont des mots.

Nos colonnes : nombres ou catégories ?

heure	nombre	19
jour de la semaine	catégorie	mardi
mois	catégorie	janvier
férié	catégorie	oui / non
zone de vacances	catégorie	A, B ou C
température	nombre	2 °C
consommation 7 jours avant	nombre	74 500 MW

Valeurs d'exemple illustratives.

Comment on l'utilise

On indique les colonnes catégorielles :

```
cat_features = [jour, mois, férié, zone]
```

- Mots lus tels quels.
- Bon résultat sans long réglage.
- Plus lent à entraîner.

Son rôle chez nous

Tester jour, mois et vacances comme de vraies catégories.

Comparaison et méthode

	XGBoost	LightGBM	CatBoost
Son nom	eXtreme Gradient Boosting	Light Gradient Boosting Machine	Categorical Boosting
Créé	2014 · Tianqi Chen	2016 · Microsoft	2017 · Yandex
Spécialité	Fiabilité · gère les trous	Vitesse · mémoire	Catégories · ordered boosting
Arbres	Niveau par niveau	Feuille par feuille	Symétriques
À surveiller	Mots $\longrightarrow$ nombres	Par-cœur si trop de feuilles	Entraînement plus lent
Chez nous	**Modèle de référence**	**Exploration rapide, fourchettes**	**Jour, mois, vacances en catégories**

Comparaison et méthode

	XGBoost	LightGBM	CatBoost
Son nom	eXtreme Gradient Boosting	Light Gradient Boosting Machine	Categorical Boosting
Créé	2014 · Tianqi Chen	2016 · Microsoft	2017 · Yandex
Spécialité	Fiabilité · gère les trous	Vitesse · mémoire	Catégories · ordered boosting
Arbres	Niveau par niveau	Feuille par feuille	Symétriques
À surveiller	Mots → nombres	Par-cœur si trop de feuilles	Entraînement plus lent
Chez nous	**Modèle de référence**	**Exploration rapide, fourchettes**	**Jour, mois, vacances en catégories**



Les indicateurs ajoutés : calendrier et météo

Une seule table · 1 ligne = 1 heure (repérée en UTC) · 2012 → 2025 · 122 736 lignes

Calendrier

Fichier du groupe (1 ligne par jour), recopié sur les 24 heures du jour.

jour, mois, saison	mardi, janvier
week-end, jour ouvré	oui / non
jour férié	oui / non
vacances scolaires	zones A, B, C
Noël, saison de chauffage	oui / non

Les indicateurs ajoutés : calendrier et météo

Une seule table · 1 ligne = 1 heure (repérée en UTC) · 2012 → 2025 · 122 736 lignes

Calendrier

Fichier du groupe (1 ligne par jour), recopié sur les 24 heures du jour.

jour, mois, saison	mardi, janvier
week-end, jour ouvré	oui / non
jour férié	oui / non
vacances scolaires	zones A, B, C
Noël, saison de chauffage	oui / non

Météo

Météo-France, 12 stations (1 par région), moyenne France pondérée par la population.

température	2 °C
rayonnement solaire	350 W/m²
ensoleillement	40 min / h
vent	4 m/s
humidité, pluie	85 %, 0,2 mm

Les indicateurs ajoutés : calendrier et météo

Une seule table · 1 ligne = 1 heure (repérée en UTC) · 2012 → 2025 · 122 736 lignes

Calendrier

Fichier du groupe (1 ligne par jour), recopié sur les 24 heures du jour.

jour, mois, saison	mardi, janvier
week-end, jour ouvré	oui / non
jour férié	oui / non
vacances scolaires	zones A, B, C
Noël, saison de chauffage	oui / non

Météo

Météo-France, 12 stations (1 par région), moyenne France pondérée par la population.

température	2 °C
rayonnement solaire	350 W/m²
ensoleillement	40 min / h
vent	4 m/s
humidité, pluie	85 %, 0,2 mm

À retenir

Températures **mesurées**, pas prévues : le score sera un peu optimiste. Calendrier à étendre à 2012 – 2022.

Notre base de données

Une seule table horaire, la même pour les trois modèles et pour tout le groupe.

Ce qu'elle contient

122 736 heures

du 1er janv. 2012 au 31 déc. 2025

1 ligne = 1 heure

repérée en heure UTC

14 heures vides

1 par an : passage à l'heure d'hiver

Notre base de données

Une seule table horaire, la même pour les trois modèles et pour tout le groupe.

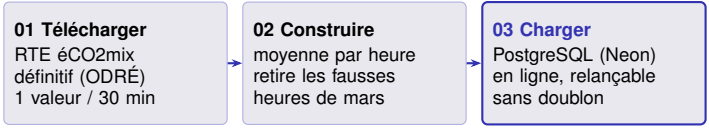
Ce qu'elle contient

122 736 heures
du 1er janv. 2012 au 31 déc. 2025

1 ligne = 1 heure
repérée en heure UTC

14 heures vides
1 par an : passage à l'heure d'hiver

Comment elle est construite : 3 scripts Python



Changements d'heure : en UTC, toujours 1 h entre deux lignes.
Mars : heure en trop retirée · octobre : heure manquante laissée vide.

Notre base de données

Une seule table horaire, la même pour les trois modèles et pour tout le groupe.

Ce qu'elle contient

122 736 heures
du 1er janv. 2012 au 31 déc. 2025

1 ligne = 1 heure
repérée en heure UTC

14 heures vides
1 par an : passage à l'heure d'hiver

Comment elle est construite : 3 scripts Python

01 Télécharger
RTE éCO2mix
définitif (ODRÉ)
1 valeur / 30 min

→

02 Construire
moyenne par heure
retire les fausses
heures de mars

→

03 Charger
PostgreSQL (Neon)
en ligne, relançable
sans doublon

Changements d'heure : en UTC, toujours 1 h entre deux lignes.
Mars : heure en trop retirée · octobre : heure manquante laissée vide.

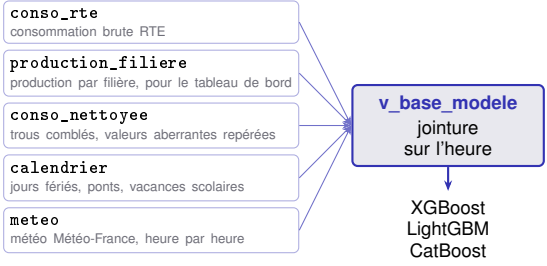
Le principe

Les données RTE sont gardées brutes, seulement mises au pas horaire : rien n'est corrigé ici. On garde aussi la prévision de la veille de RTE, la référence que nos modèles devront battre.

Comment la base sert au projet

Une table par source de données ; une vue assemble tout, heure par heure, pour les modèles.

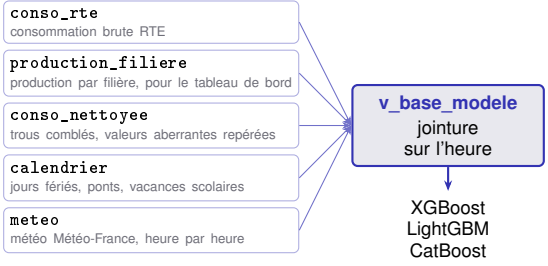
Une table par source, une vue pour tout réunir



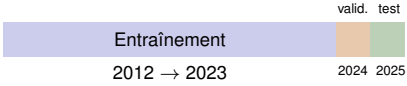
Comment la base sert au projet

Une table par source de données ; une vue assemble tout, heure par heure, pour les modèles.

Une table par source, une vue pour tout réunir



Le même découpage pour tous



105 192 h pour apprendre, 8 784 h pour régler, 8 760 h pour le verdict final.

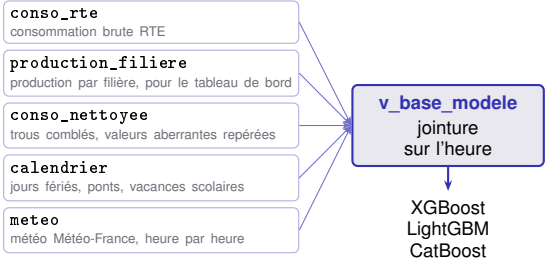
Les règles de la base

– Rien d'inconnu la veille : conso passée d'au moins 48 h.
– Météo mesurée : score un peu optimiste, on le signalera.
– Production par filière : tableau de bord seulement.

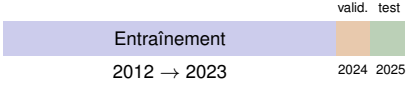
Comment la base sert au projet

Une table par source de données ; une vue assemble tout, heure par heure, pour les modèles.

Une table par source, une vue pour tout réunir



Le même découpage pour tous



105 192 h pour apprendre, 8 784 h pour régler, 8 760 h pour le verdict final.

Les règles de la base

- Rien d'inconnu la veille : conso passée d'au moins 48 h.
- Météo mesurée : score un peu optimiste, on le signalera.
- Production par filière : tableau de bord seulement.

Notre vision du projet

Une seule source pour tout le groupe : mêmes heures, même découpage, donc des scores comparables. À partir de ces tables, on construit maintenant les variables des modèles.

Target encoding sur nos données

On remplace *Férié* (oui / non) par la conso moyenne de chaque catégorie.

1. Le principe

Un modèle lit des **nombres**. « oui » et « non » sont du texte.

On les remplace par la **conso moyenne** de leur catégorie :

$$\mathrm{enc}(c) = \frac{1}{n_c} \sum_{i\,:\,x_i = c} y_i$$

n_c : nb de lignes de c ; y : conso.

Target encoding sur nos données

On remplace *Férié* (oui / non) par la conso moyenne de chaque catégorie.

1. Le principe

Un modèle lit des **nombres**. « oui » et « non » sont du texte.

On les remplace par la **conso moyenne** de leur catégorie :

$$\mathrm{enc}(c) = \frac{1}{n_c} \sum_{i\,:\,x_i = c} y_i$$

n_c : nb de lignes de c ; y : conso.

2. Sur notre extrait (4 lignes)

Date	Férié	Conso (MW)	Encodé
14 janv. 03 h	non	56 200	**66 867**
14 janv. 19 h	non	78 100	**66 867**
19 janv. 19 h	non	66 300	**66 867**
15 août 13 h	oui	36 500	**36 500**

non : $\dfrac{56\,200 + 78\,100 + 66\,300}{3} = 66\,867$

oui : $\dfrac{36\,500}{1} = 36\,500$

Target encoding sur nos données

On remplace *Férié* (oui / non) par la conso moyenne de chaque catégorie.

1. Le principe

Un modèle lit des **nombres**. « oui » et « non » sont du texte.

On les remplace par la **conso moyenne** de leur catégorie :

$$\mathrm{enc}(c) = \frac{1}{n_c} \sum_{i\,:\,x_i = c} y_i$$

n_c : nb de lignes de c ; y : conso.

2. Sur notre extrait (4 lignes)

Date	Férié	Conso (MW)	Encodé
14 janv. 03 h	non	56 200	**66 867**
14 janv. 19 h	non	78 100	**66 867**
19 janv. 19 h	non	66 300	**66 867**
15 août 13 h	oui	36 500	**36 500**

non : $\dfrac{56\,200 + 78\,100 + 66\,300}{3} = 66\,867$

oui : $\dfrac{36\,500}{1} = 36\,500$

3. Deux limites

! *oui* n'a qu'**1 ligne** : sa moyenne n'est pas fiable.

! Calcul sur **2012→2023 seulement**, jamais avec 2024 ni 2025.

À retenir

La colonne *Férié* (texte) devient un nombre : **66 867** ou **36 500**. Mais une moyenne sur 1 ligne est fragile : c'est le rôle du lissage (slide suivante).

Lissage et règle d'utilisation

Peu d'exemples : on se rapproche de la moyenne générale.

1. La formule

$$\mathrm{enc}(c) = w_c\,\bar{y}_c + (1 - w_c)\,\bar{y}$$

$$w_c = \frac{n_c}{n_c + m}$$

$\bar{y}_c$: moyenne de c
$\bar{y}$: moyenne générale $= 59\,275$
m : réglage (ici $m = 2$)

n_c petit $\Rightarrow$ w_c petit $\Rightarrow$ on suit $\bar{y}$.

Lissage et règle d'utilisation

Peu d'exemples : on se rapproche de la moyenne générale.

1. La formule

$$\mathrm{enc}(c) = w_c\,\bar{y}_c + (1 - w_c)\,\bar{y}$$

$$w_c = \frac{n_c}{n_c + m}$$

$\bar{y}_c$: moyenne de c
$\bar{y}$: moyenne générale $= 59\,275$
m : réglage (ici $m = 2$)

n_c petit $\Rightarrow$ w_c petit $\Rightarrow$ on suit $\bar{y}$.

2. Résultat sur l'extrait

Férié	n_c	w_c	Brut	Lissé
non	3	0,60	66 867	**63 830**
oui	1	0,33	36 500	**51 683**
inconnue	0	0	–	**59 275**

oui : $\frac{1}{3} \times 36\,500 + \frac{2}{3} \times 59\,275 = 51\,683$

Extrait de 4 lignes : sur la vraie base, oui aura bien plus de lignes.

Lissage et règle d'utilisation

Peu d'exemples : on se rapproche de la moyenne générale.

1. La formule

$$\mathrm{enc}(c) = w_c\,\bar{y}_c + (1 - w_c)\,\bar{y}$$

$$w_c = \frac{n_c}{n_c + m}$$

$\bar{y}_c$: moyenne de c
$\bar{y}$: moyenne générale $= 59\,275$
m : réglage (ici $m = 2$)

n_c petit $\Rightarrow$ w_c petit $\Rightarrow$ on suit $\bar{y}$.

2. Résultat sur l'extrait

Férié	n_c	w_c	Brut	Lissé
non	3	0,60	66 867	**63 830**
oui	1	0,33	36 500	**51 683**
inconnue	0	0	–	**59 275**

oui : $\frac{1}{3} \times 36\,500 + \frac{2}{3} \times 59\,275 = 51\,683$

Extrait de 4 lignes : sur la vraie base, oui aura bien plus de lignes.

3. Dans le pipeline



Entraînement : on calcule.
2024 et 2025 : on applique seulement.

CatBoost

CatBoost le fait déjà seul, sans triche : `cat_features = [jour, mois, férié, zone]`. Avec XGBoost ou LightGBM, on encode nous-mêmes.